

Bayesian Learning, Concept Learning, and EM Algorithm

(Based on Tom Mitchell, Chapter 6)

Contents

1 Bayesian Learning: Overview	3
2 Bayes Theorem	3
2.1 Maximum a Posteriori (MAP)	3
2.2 Maximum Likelihood (ML)	5
3 Bayes Theorem: A Very Practical Medical Example	5
4 Relation to Concept Learning	6
5 Minimum Description Length (MDL): Choosing Simplicity	7
6 Naive Bayes Classifier	8
7 The Bayesian Framework for Machine Learning	9
7.1 1. Bayes' Theorem for Model Weights	10
8 Gibbs Algorithm	14
9 Bayesian Belief Network (BBN)	15
9.1 1. Definition	15
9.2 2. Diagrammatic Representation (Example Network)	16
9.3 3. Dataset Representation (Input/Output View)	16
9.4 4. Conditional Probability Tables (CPT)	17
9.5 5. Conditional Independence	17
10 Inference in Bayesian Networks	17
10.1 Example: Computing $P(R = 1 \mid W = 1)$	17
11 7. General Inference Formula	18
12 8. Summary	19
13 Expectation Maximization (EM) for Gaussian Mixture Models	19
13.1 Model Setup	19
13.2 Example Dataset	20
13.3 Graphical Interpretation	20

14 EM Algorithm	20
14.1 E-Step: Compute Responsibilities	20
14.2 M-Step: Update All Parameters	21
14.3 Numerical Update Using Example	21
14.4 Complete EM Iteration	22
15 Summary	22

1 Bayesian Learning: Overview

Bayesian learning provides a **probabilistic framework** for reasoning about hypotheses in the presence of uncertainty. Instead of selecting a single hypothesis, it maintains a *distribution over hypotheses*.

Two major roles:

- **Practical algorithms:** Naive Bayes, Bayesian Networks, EM
- **Conceptual gold standard:** Optimal decision making under uncertainty

This framework formalizes:

- Prior knowledge
- Evidence from data
- Occam's Razor

2 Bayes Theorem

Bayes theorem relates prior belief and observed evidence:

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}$$

- $P(h)$: Prior probability of hypothesis
- $P(D | h)$: Likelihood of data
- $P(h | D)$: Posterior probability
- $P(D)$: Evidence (normalizing constant)

In Bayesian learning, hypotheses are treated as random variables. Given a set of training data D , the goal is to determine which hypothesis is most probable after observing the data.

2.1 Maximum a Posteriori (MAP)

Bayes' theorem gives the posterior probability of a hypothesis h as

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)} \tag{1}$$

Since $P(D)$ is constant for all hypotheses, maximizing the posterior is equivalent to maximizing the numerator. Thus, the **Maximum a Posteriori (MAP)** hypothesis is defined as

$$h_{MAP} = \arg \max_{h \in H} P(D | h) P(h) \tag{2}$$

$$\text{Posterior} = \text{Likelihood} \times \text{Prior}$$

MAP combines two important components:

- **Likelihood** $P(D|h)$: how well hypothesis h explains the observed data
- **Prior** $P(h)$: how plausible or preferred hypothesis h is before seeing the data

Hence, MAP naturally incorporates prior knowledge and provides a formal realization of Occam's Razor, preferring simpler or more plausible hypotheses unless the data strongly supports a more complex one.

Bernoulli Distribution (Binary Classification)

For a single trial $y \in \{0, 1\}$ with success probability p :

$$p(y | x) = p^y(1 - p)^{1-y}$$

The Negative Log-Likelihood (NLL) is:

$$\mathcal{L}_{\text{NLL}} = -\log p(y | x) = -[y \log p + (1 - y) \log(1 - p)]$$

Binomial Distribution (Multiple Bernoulli Trials) For n independent trials with y successes:

$$p(y | x) = \binom{n}{y} p^y (1 - p)^{n-y}$$

Example: Coin Bias (MAP)

Consider two hypotheses about a coin:

- h_1 : the coin is fair, $P(\text{Heads}) = 0.5$
- h_2 : the coin is biased, $P(\text{Heads}) = 0.9$

Suppose we observe D : 9 heads and 1 tail in 10 tosses.

The likelihoods are:

$$P(D | h_1) = \binom{10}{9} (0.5)^{10} \tag{3}$$

$$P(D | h_2) = \binom{10}{9} (0.9)^9 (0.1) \tag{4}$$

Assume the prior belief that coins are usually fair:

$$P(h_1) = 0.9, \quad P(h_2) = 0.1$$

The MAP scores are:

$$P(D | h_1)P(h_1) \approx 0.0098 \times 0.9 \tag{5}$$

$$P(D | h_2)P(h_2) \approx 0.387 \times 0.1 \tag{6}$$

Since $P(D | h_2)P(h_2)$ is larger, the MAP hypothesis is h_2 .

2.2 Maximum Likelihood (ML)

If all hypotheses are assumed to be equally likely a priori, that is,

$$P(h_i) = P(h_j) \quad \forall h_i, h_j \in H, \quad (7)$$

then the prior term does not affect the optimization. In this case, MAP reduces to **Maximum Likelihood (ML)** estimation:

$$h_{ML} = \arg \max_{h \in H} P(D | h) \quad (8)$$

ML selects the hypothesis that makes the observed data most probable, without considering prior preferences or model complexity.

Example: Coin Bias (ML)

Using the same likelihoods as above,

$$P(D | h_2) > P(D | h_1)$$

so ML also selects the biased coin hypothesis h_2 .

Overfitting with ML

Because ML ignores prior knowledge, it tends to prefer more complex hypotheses that fit the training data extremely well. This can lead to overfitting, especially when the hypothesis space is large.

MAP mitigates this problem by assigning lower prior probability to overly complex hypotheses.

Key Differences Between ML and MAP

Maximum Likelihood (ML)	Maximum a Posteriori (MAP)
Uses only training data	Uses training data and prior knowledge
$h_{ML} = \arg \max P(D h)$	$h_{MAP} = \arg \max P(D h)P(h)$
Prone to overfitting	Regularized by the prior
Purely data-driven	Bayesian and principled

3 Bayes Theorem: A Very Practical Medical Example

Bayes' theorem answers a very natural question:

"If my test result is positive, what is the real chance I actually have the disease?"

Step 1: Given Information

- Only 0.8% of people have cancer:

$$P(C) = 0.008$$

- If someone has cancer, the test correctly detects it 98% of the time:

$$P(+|C) = 0.98$$

- If someone is healthy, the test correctly says negative 97% of the time:

$$P(-|\neg C) = 0.97$$

So false positive rate is:

$$P(+|\neg C) = 0.03$$

Step 2: Think in Real Numbers (1,000,000 people)

Instead of jumping into formulas, let's imagine a city of 1,000,000 people.

- 0.8% have cancer $\rightarrow$ 8,000 people
- 992,000 are healthy

Now apply test accuracy:

- 98% of 8,000 $\rightarrow$ 7,840 true positives
- 3% of 992,000 $\rightarrow$ 29,760 false positives

Total positive results:

$$37,600 = 7,840 + 29,760$$

Now the real question:

$$P(C|+) = \frac{7,840}{37,600} \approx 0.21$$

So even after testing positive, the chance of actually having cancer is only about 21%.

Big Lesson

When a disease is rare, even a very accurate test produces many false alarms.

4 Relation to Concept Learning

Basic Idea

- X = All possible examples
- H = All possible rules (hypotheses)
- D = Observed training data

Consistency Idea

If a hypothesis makes a mistake on training data, we eliminate it.

$$P(D|h) = \begin{cases} 1 & \text{if } h \text{ fits the data perfectly} \\ 0 & \text{otherwise} \end{cases}$$

So learning becomes:

“Keep only the hypotheses that agree with what we observed.”

Simple Example

Suppose we believe:

“Students who study between 2 and 5 hours pass.”

Now we observe:

A student studied 3 hours and passed.

Any rule that includes 3 hours remains possible. Any rule that excludes 3 hours is removed.

All remaining consistent rules are equally likely.

This set is called the **Version Space**.

$$P(h|D) = \frac{1}{\text{Number of consistent hypotheses}}$$

5 Minimum Description Length (MDL): Choosing Simplicity

MDL follows a very natural human principle:

Prefer the simplest explanation that works.

Idea

Total cost =

Model size + Mistakes it makes

$$h_{MDL} = \arg \min_h [L(h) + L(D|h)]$$

Example: Decision Trees

Imagine two trees:

- Very large tree (100 rules), zero training error.
- Small tree (10 rules), makes 2 mistakes.

Costs:

$$\text{Large tree cost} = 100 + 0 = 100$$

$$\text{Small tree cost} = 10 + 2 = 12$$

MDL prefers the small tree.

Why This Makes Sense

A very complex model may memorize noise. A simpler model generalizes better.

Connection to MAP

Information theory says:

$$\text{Code length} = -\log P(\text{event})$$

So maximizing probability:

$$\max P(D|h)P(h)$$

is the same as minimizing:

$$-\log P(h) - \log P(D|h)$$

Which becomes:

$$L(h) + L(D|h)$$

Therefore: MAP learning and MDL are mathematically equivalent ideas.

6 Naive Bayes Classifier

Assume conditional independence between attributes a_i given the class label y_j :

$$P(a_1, \dots, a_n | y_j) = \prod_i P(a_i | y_j) \quad (9)$$

The Naive Bayes classification y_{NB} is determined by the maximum a posteriori (MAP) decision rule:

$$y_{NB} = \arg \max_{y_j} P(y_j) \prod_i P(a_i | y_j) \quad (10)$$

In this example, we predict the target class **Play** (*Yes* or *No*) based on weather observations.

1. Training Dataset

The following table represents the training data used to estimate the prior and conditional probabilities.

Outlook	Temperature	Humidity	Wind	Play (Class)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Table 1: The PlayTennis Training Dataset

2. New Instance to Classify

We are given a new observation $\mathbf{x}$ and must predict the most likely class y_{MAP} :

$$\mathbf{x} = \langle \text{Outlook}=\text{Sunny}, \text{Temp}=\text{Cool}, \text{Humidity}=\text{High}, \text{Wind}=\text{Strong} \rangle$$

We find the class y_j that maximizes:

$$y_{NB} = \arg \max_{y_j \in \{Yes, No\}} P(y_j) \prod_i P(a_i | y_j) \quad (11)$$

3. Step-by-Step Computation

Prior Probabilities:

Total instances $n = 14$.

- $P(Yes) = 9/14 \approx 0.643$
- $P(No) = 5/14 \approx 0.357$

Conditional Probabilities (Likelihoods):

Based on the frequencies in the training table:

- $P(\text{Sunny}|Yes) = 2/9, \quad P(\text{Sunny}|No) = 3/5$
- $P(\text{Cool}|Yes) = 3/9, \quad P(\text{Cool}|No) = 1/5$
- $P(\text{High}|Yes) = 3/9, \quad P(\text{High}|No) = 4/5$
- $P(\text{Strong}|Yes) = 3/9, \quad P(\text{Strong}|No) = 3/5$

Posterior Probability for Class $y_1 = Yes$:

$$\begin{aligned} P(Yes|\mathbf{x}) &\propto P(Yes) \cdot P(\text{Sunny}|Yes) \cdot P(\text{Cool}|Yes) \cdot P(\text{High}|Yes) \cdot P(\text{Strong}|Yes) \\ &\propto \frac{9}{14} \cdot \left(\frac{2}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \right) \\ &= 0.643 \cdot 0.0082 = \mathbf{0.0053} \end{aligned}$$

Posterior Probability for Class $y_2 = No$:

$$\begin{aligned} P(No|\mathbf{x}) &\propto P(No) \cdot P(\text{Sunny}|No) \cdot P(\text{Cool}|No) \cdot P(\text{High}|No) \cdot P(\text{Strong}|No) \\ &\propto \frac{5}{14} \cdot \left(\frac{3}{5} \cdot \frac{1}{5} \cdot \frac{4}{5} \cdot \frac{3}{5} \right) \\ &= 0.357 \cdot 0.0576 = \mathbf{0.0206} \end{aligned}$$

4. Conclusion

Since $P(No|\mathbf{x}) > P(Yes|\mathbf{x})$, the Naive Bayes classifier assigns the instance to the class **No**.

7 The Bayesian Framework for Machine Learning

In Bayesian Learning, we treat the process of training a model not as a simple optimization of a single value, but as a process of **probabilistic inference**. We use a dataset $D = \{(x_i, y_i)\}_{i=1}^n$ to update our beliefs about the model parameters (weights) $\mathbf{w}$.

7.1 1. Bayes' Theorem for Model Weights

The fundamental equation that governs Bayesian Learning is:

$$P(\mathbf{w}|D) = \frac{P(D|\mathbf{w})P(\mathbf{w})}{P(D)} \quad (12)$$

Where:

- $P(w|D) = P(w|x, y)$ is the **Posterior Distribution**: The updated probability distribution of the weights after seeing the training data.
- $P(D|w) \approx P(y|x, w)$ is the **Likelihood**: The probability that the observed labels y were generated by inputs x given weights $\mathbf{w}$.
- $P(\mathbf{w})$ is the **Prior**: Our initial assumption about the distribution of weights (e.g., a Gaussian distribution).
- $P(D)$ is the **Evidence**: A normalization constant calculated as $\int P(D|\mathbf{w})P(\mathbf{w})d\mathbf{w}$.

Discriminative Bayesian Learning: Weight Inference

In this scenario, we use a Logistic Regression model to determine the probability of a student passing ($y = 1$) based on study hours (x). We compare two competing hypotheses for the weight parameter w .

1. Training Dataset (D)

The dataset consists of four observations representing hours studied and the binary result.

Student	Hours Studied (x)	Result (y)
1	1	0 (Fail)
2	2	0 (Fail)
3	5	1 (Pass)
4	6	1 (Pass)

Table 2: Exam Results Dataset

2. Hypotheses and Priors

We consider two possible values for the weight w , starting with an unbiased prior:

- $w_1 = 0.1$ (Weak correlation): $P(w_1) = 0.5$
- $w_2 = 1.0$ (Strong correlation): $P(w_2) = 0.5$

3. Likelihood Computation

The probability of a single binary outcome $y \in \{0, 1\}$ is given by:

$$P(Y = y|p) = p^y(1 - p)^{1-y} \quad (13)$$

In a discriminative model where $\hat{y}_i = \sigma(wx_i)$, the likelihood of an observation is:

$$P(y_i|x_i, w) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i} \quad (14)$$

The discriminative likelihood for the dataset is the product of the individual Bernoulli trials:

$$P(D|w) = \prod_{i=1}^n \sigma(wx_i)^{y_i} \cdot (1 - \sigma(wx_i))^{1-y_i} \quad (15)$$

Likelihood for $w_1 = 0.1$:

$$\begin{aligned} P(D|w_1) &= (1 - \sigma(0.1)) \cdot (1 - \sigma(0.2)) \cdot \sigma(0.5) \cdot \sigma(0.6) \\ &\approx 0.475 \cdot 0.450 \cdot 0.622 \cdot 0.646 \approx \mathbf{0.0859} \end{aligned}$$

Likelihood for $w_2 = 1.0$:

$$\begin{aligned} P(D|w_2) &= (1 - \sigma(1.0)) \cdot (1 - \sigma(2.0)) \cdot \sigma(5.0) \cdot \sigma(6.0) \\ &\approx 0.269 \cdot 0.119 \cdot 0.993 \cdot 0.997 \approx \mathbf{0.0317} \end{aligned}$$

4. Posterior Distribution Computation

Using Bayes' Rule, we update our belief for each weight:

$$P(w_j|D) = \frac{P(D|w_j)P(w_j)}{\sum_k P(D|w_k)P(w_k)} \quad (16)$$

Step 1: Calculate Numerators

- Numerator for w_1 : $0.0859 \times 0.5 = 0.04295$
- Numerator for w_2 : $0.0317 \times 0.5 = 0.01585$

Step 2: Evidence Normalization

$$P(D) = 0.04295 + 0.01585 = 0.0588$$

Step 3: Final Posteriors

- $P(w_1|D) = \frac{0.04295}{0.0588} \approx \mathbf{0.73}$
- $P(w_2|D) = \frac{0.01585}{0.0588} \approx \mathbf{0.27}$

Maximum Likelihood Estimation (MLE) in Linear Regression

In this example, we derive the optimal weights for a linear model by maximizing the likelihood of the observed data under the assumption of Gaussian noise.

1. The Probabilistic Model

We assume that the target variable y_i is a linear function of the input x_i with added Gaussian noise ϵ_i :

$$y_i = wx_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (17)$$

This implies that y_i itself follows a Gaussian distribution centered at the model's prediction:

$$P(y_i|x_i, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - wx_i)^2}{2\sigma^2}\right) \quad (18)$$

2. The Likelihood Function

For a dataset $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, assuming the observations are independent and identically distributed (i.i.d.), the total likelihood is the product of individual densities:

$$L(w) = \prod_{i=1}^n P(y_i|x_i, w) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - wx_i)^2}{2\sigma^2}\right) \quad (19)$$

3. Log-Likelihood Derivation

To simplify the optimization, we take the natural logarithm ($\ln$) to obtain the Log-Likelihood $\ell(w)$:

$$\ell(w) = \ln\left(\prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - wx_i)^2}{2\sigma^2}\right)\right) \quad (20)$$

$$= \sum_{i=1}^n \left[\ln\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{(y_i - wx_i)^2}{2\sigma^2} \right] \quad (21)$$

$$= n \ln\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - wx_i)^2 \quad (22)$$

4. Connection to Mean Squared Error (MSE)

To find the Maximum Likelihood Estimator $\hat{w}_{MLE}$, we maximize $\ell(w)$. Since the first term is constant and the coefficient in the second term is negative, maximizing $\ell(w)$ is equivalent to **minimizing** the sum of squared differences:

$$\hat{w}_{MLE} = \arg \min_w \sum_{i=1}^n (y_i - wx_i)^2 \quad (23)$$

This proves that under Gaussian assumptions, MLE is equivalent to the **Ordinary Least Squares (OLS)** method used in Mean Squared Error minimization.

5. Solving for the Optimal Weight

Setting the derivative of the summation with respect to w to zero:

$$\frac{d}{dw} \sum_{i=1}^n (y_i - wx_i)^2 = 0 \quad (24)$$

$$-2 \sum_{i=1}^n x_i (y_i - wx_i) = 0 \quad (25)$$

$$\sum_{i=1}^n x_i y_i - w \sum_{i=1}^n x_i^2 = 0 \quad (26)$$

$$\hat{w}_{MLE} = \frac{\sum x_i y_i}{\sum x_i^2} \quad (27)$$

Equivalence of MLE and Cross-Entropy Loss

In binary classification, we model the probability of a label $y_i \in \{0, 1\}$ given an input x_i and model parameters w .

1. The Bernoulli Likelihood

We assume each observation y_i is a realization of a Bernoulli random variable with probability $p_i = \sigma(w^T x_i)$. The probability mass function is:

$$P(y_i | x_i, w) = p_i^{y_i} (1 - p_i)^{1-y_i} \quad (28)$$

2. The Likelihood of the Dataset

For n independent observations, the likelihood $L(w)$ is the product of individual probabilities:

$$L(w) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} \quad (29)$$

3. Log-Likelihood Transformation

To find the Maximum Likelihood Estimator, we maximize the log-likelihood $\ell(w)$:

$$\ell(w) = \ln \left(\prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} \right) \quad (30)$$

$$= \sum_{i=1}^n \ln (p_i^{y_i} (1 - p_i)^{1-y_i}) \quad (31)$$

$$= \sum_{i=1}^n [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)] \quad (32)$$

4. Negative Log-Likelihood and Cross-Entropy

In machine learning, we prefer to minimize a loss function rather than maximize a likelihood. We define the **Negative Log-Likelihood (NLL)**:

$$\text{NLL}(w) = -\ell(w) = -\sum_{i=1}^n [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)] \quad (33)$$

This expression is exactly the formula for **Binary Cross-Entropy (BCE)** loss.

5. Conclusion

Minimizing the Cross-Entropy loss is mathematically identical to maximizing the Likelihood of the observed class labels under a Bernoulli distribution assumption. Thus:

$$\hat{w}_{MLE} = \arg \min_w \text{Cross-Entropy}(y, p) \quad (34)$$

Summary

Maximum Likelihood selects the hypothesis that best explains the observed data, while Maximum a Posteriori selects the hypothesis that best balances data fit and prior belief. When priors are uniform, MAP and ML are equivalent; otherwise, MAP provides a more robust and realistic learning criterion.

8 Gibbs Algorithm

1. Motivation: Goal is to Approximate the Bayes optimal classifier.

The Bayes optimal classifier predicts by averaging over all hypotheses:

$$\hat{y} = \arg \max_y \sum_{h \in H} P(y|x, h) P(h|D)$$

Bayes optimal $\rightarrow$ integrates over all hypothesis

Naive Bayes $\rightarrow$ uses one best hypothesis estimate

Although optimal, this requires summing over possibly very large hypothesis spaces.

The Gibbs algorithm provides a simple approximation.

2. Algorithm

1. Sample a hypothesis from the posterior:

$$h \sim P(h|D)$$

2. Predict using that hypothesis:

$$\hat{y} = h(x)$$

Instead of averaging all models, we randomly choose one plausible model.

3. Intuition

Hypotheses with higher posterior probability are more likely to be selected. Thus, on average, good hypotheses dominate predictions.

4. Example

Suppose:

$$P(h_1|D) = 0.6, \quad P(h_2|D) = 0.3, \quad P(h_3|D) = 0.1$$

Gibbs selects:

- h_1 with probability 0.6
- h_2 with probability 0.3
- h_3 with probability 0.1

Over many predictions, this behavior approximates Bayes optimal.

5. Theoretical Guarantee

$$E[\text{error}_{Gibbs}] \leq 2E[\text{error}_{Bayes}]$$

This means Gibbs has at most twice the expected error of the Bayes optimal classifier.

Summary

- Bayes optimal = weighted vote of all hypotheses.
- Gibbs = randomly sample one hypothesis from the posterior.
- Computationally simpler with strong theoretical guarantee.

9 Bayesian Belief Network (BBN)

9.1 1. Definition

A **Bayesian Belief Network (BBN)** is a probabilistic graphical model that represents a set of random variables and their conditional dependencies using a **Directed Acyclic Graph (DAG)**.

- Each node represents a random variable.
- Each directed edge represents conditional dependence.
- Each node is associated with a Conditional Probability Table (CPT).

Let the set of variables be $X = \{X_1, X_2, \dots, X_n\}$. If $\text{Pa}(X_i)$ denotes the parents of X_i in the DAG, then the joint probability distribution factorizes as:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Pa}(X_i))$$

This compact factorization avoids the exponential complexity of full joint probability tables.

9.2 2. Diagrammatic Representation (Example Network)

Consider a simple weather dataset with variables:

- R : Rain (Yes/No)
- S : Sprinkler (On/Off)
- W : WetGrass (Yes/No)

The causal structure is:

$$R \rightarrow W \leftarrow S$$

This means:

- Rain influences whether the grass is wet.
- Sprinkler also influences wet grass.

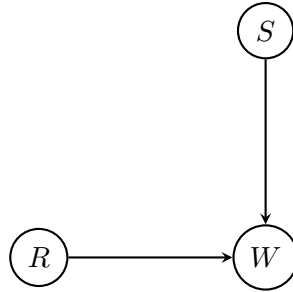


Figure 1: Bayesian Belief Network: Rain, Sprinkler and WetGrass

The joint distribution factorizes as:

$$P(R, S, W) = P(R) P(S) P(W | R, S)$$

9.3 3. Dataset Representation (Input/Output View)

Suppose we observe the following dataset:

Rain (R)	Sprinkler (S)	WetGrass (W)
1	1	1
1	0	1
0	1	1
0	0	0
1	1	1
0	1	1

Here:

- Inputs: (R, S)
- Target Output: W

Thus the model predicts:

$$W = f(R, S)$$

but probabilistically using:

$$P(W | R, S)$$

9.4 4. Conditional Probability Tables (CPT)

From the dataset, we estimate probabilities.

Prior probabilities

$$P(R = 1) = \frac{3}{6} = 0.5, \quad P(S = 1) = \frac{4}{6} \approx 0.67$$

Conditional probability

$$P(W = 1 \mid R, S) = \begin{cases} 1.0 & (R = 1, S = 1) \\ 1.0 & (R = 1, S = 0) \\ 1.0 & (R = 0, S = 1) \\ 0.0 & (R = 0, S = 0) \end{cases}$$

Thus,

$$P(R, S, W) = P(R)P(S)P(W \mid R, S)$$

9.5 5. Conditional Independence

In this network:

$$R \perp S$$

but given W :

$$R \not\perp S \mid W$$

This phenomenon is known as **explaining away**: if grass is wet, either rain or sprinkler could be responsible.

10 Inference in Bayesian Networks

Inference means computing posterior probabilities given observed evidence.

10.1 Example: Computing $P(R = 1 \mid W = 1)$

We want to compute the probability that it rained given that the grass is wet:

$$P(R = 1 \mid W = 1)$$

Using Bayes rule in conditional form:

$$P(R = 1 \mid W = 1) = \frac{P(W = 1 \mid R = 1) P(R = 1)}{P(W = 1)}$$

Since W depends on both R and S , we marginalize over the hidden variable S .

Step 1: Compute $P(W = 1 \mid R = 1)$

$$P(W = 1 \mid R = 1) = \sum_{s \in \{0,1\}} P(W = 1 \mid R = 1, S = s) P(S = s)$$

$$= P(W = 1 \mid 1, 1)P(S = 1) + P(W = 1 \mid 1, 0)P(S = 0)$$

Substitute values:

$$= (1)(0.67) + (1)(0.33) = 1.0$$

Thus,

$$P(W = 1 \mid R = 1) = 1$$

Step 2: Compute $P(W = 1)$

We marginalize over both R and S :

$$P(W = 1) = \sum_{r \in \{0,1\}} \sum_{s \in \{0,1\}} P(W = 1 \mid r, s) P(r) P(s)$$

$$= P(1, 1, 1) + P(1, 0, 1) + P(0, 1, 1) + P(0, 0, 1)$$

$$= (0.5)(0.67)(1) + (0.5)(0.33)(1) + (0.5)(0.67)(1) + (0.5)(0.33)(0)$$

$$= 0.335 + 0.165 + 0.335 + 0 = 0.835$$

Step 3: Final Posterior

$$P(R = 1 \mid W = 1) = \frac{P(W = 1 \mid R = 1) P(R = 1)}{P(W = 1)} = \frac{(1)(0.5)}{0.835} \approx 0.598$$

Interpretation: If the grass is wet, the probability that it rained is approximately 59.8%. This result reflects uncertainty because the sprinkler could also cause the grass to be wet.

11 7. General Inference Formula

For any query variable Y and evidence $\mathbf{E}$:

$$P(Y \mid \mathbf{E}) = \frac{\sum_{\mathbf{Z}} \prod_i P(X_i \mid \text{Pa}(X_i))}{\sum_{Y, \mathbf{Z}} \prod_i P(X_i \mid \text{Pa}(X_i))}$$

where $\mathbf{Z}$ are hidden variables.

12 8. Summary

A Bayesian Belief Network:

- Represents variables using a DAG
- Encodes joint distribution as:

$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Pa}(X_i))$$

- Supports probabilistic prediction and inference

It is widely used for:

- Medical diagnosis
- Fault detection
- Decision support systems
- Causal reasoning in AI

13 Expectation Maximization (EM) for Gaussian Mixture Models

Expectation–Maximization (EM) is used to estimate parameters of a mixture of Gaussians when the component assignments are hidden (latent variables).

13.1 Model Setup

Assume data points $X = \{x_1, x_2, \dots, x_n\}$ are generated from a mixture of K Gaussian distributions. Each point belongs to an unknown component $z_i \in \{1, \dots, K\}$.

Hidden indicator variable:

$$z_{ij} = \begin{cases} 1 & \text{if } x_i \text{ comes from Gaussian } j \\ 0 & \text{otherwise} \end{cases}$$

Mixture likelihood:

$$P(x_i) = \sum_{j=1}^K \pi_j \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)$$

where

- π_j = mixing coefficient ($\sum_j \pi_j = 1$),
- μ_j = mean of component j ,
- σ_j^2 = variance of component j .

13.2 Example Dataset

Consider 1D data:

$$X = \{1, 2, 3, 8, 9, 10\}, \quad K = 2$$

Initial parameters:

$$\mu_1 = 2, \quad \mu_2 = 9, \quad \sigma_1^2 = \sigma_2^2 = 1, \quad \pi_1 = \pi_2 = 0.5$$

These represent two clusters:

- Cluster 1 near small values
- Cluster 2 near large values

13.3 Graphical Interpretation



Figure 2: Data generated from two Gaussian components

14 EM Algorithm

EM alternates between:

- E-step: estimate hidden memberships
- M-step: update parameters $(\pi_j, \mu_j, \sigma_j^2)$

Likelihood never decreases during iterations.

14.1 E-Step: Compute Responsibilities

Soft assignment (posterior membership):

$$\gamma_{ij} = E[z_{ij}] = \frac{\pi_j \mathcal{N}(x_i | \mu_j, \sigma_j^2)}{\sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \sigma_k^2)}$$

Meaning of Soft Assignment γ_{ij} The quantity

$$\gamma_{ij} = E[z_{ij}] = P(z_i = j | x_i, \theta)$$

denotes the **degree of membership (responsibility)** of the i^{th} data point x_i to the j^{th} Gaussian component.

Thus, $\gamma_{ij} \in [0, 1]$ and

$$\sum_{j=1}^K \gamma_{ij} = 1,$$

meaning each data point is softly assigned to all components with different probabilities.

Interpretation:

γ_{ij} = “Probability that Gaussian component j generated data point x_i .”

Example: For $x_1 = 1$

$$\gamma_{11} = \frac{e^{-\frac{(1-2)^2}{2}}}{e^{-\frac{(1-2)^2}{2}} + e^{-\frac{(1-9)^2}{2}}} \approx 1$$

Thus x_1 mostly belongs to Gaussian 1.

For $x_5 = 9$:

$$\gamma_{52} \approx 1$$

So it belongs to Gaussian 2.

14.2 M-Step: Update All Parameters

We must update **all** parameters, not just means.

1. Update Mixing Coefficients

$$\pi_j^{new} = \frac{1}{n} \sum_{i=1}^n \gamma_{ij}$$

Interpretation: fraction of data assigned to component j .

2. Update Means

$$\mu_j^{new} = \frac{\sum_{i=1}^n \gamma_{ij} x_i}{\sum_{i=1}^n \gamma_{ij}}$$

3. Update Variances

$$\sigma_j^{2new} = \frac{\sum_{i=1}^n \gamma_{ij} (x_i - \mu_j)^2}{\sum_{i=1}^n \gamma_{ij}}$$

14.3 Numerical Update Using Example

From responsibilities:

$$\sum_i \gamma_{i1} \approx 3, \quad \sum_i \gamma_{i2} \approx 3$$

Mixing coefficients:

$$\pi_1 = \frac{3}{6} = 0.5, \quad \pi_2 = \frac{3}{6} = 0.5$$

Means:

$$\mu_1 = \frac{1 + 2 + 3}{3} = 2, \quad \mu_2 = \frac{8 + 9 + 10}{3} = 9$$

Variances:

$$\sigma_1^2 = \frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3} = \frac{2}{3}$$
$$\sigma_2^2 = \frac{(8-9)^2 + (9-9)^2 + (10-9)^2}{3} = \frac{2}{3}$$

Thus parameters move toward cluster statistics.

14.4 Complete EM Iteration

1. Initialize π_j, μ_j, σ_j^2
2. **E-step:** compute responsibilities γ_{ij}
3. **M-step:** update π_j, μ_j, σ_j^2
4. Repeat until convergence

This process gradually separates the dataset into probabilistic clusters, while updating both cluster centers, spreads, and prior probabilities.

15 Summary

- Bayesian learning = probabilistic reasoning over uncertainty
- MAP and MDL are mathematically related objectives
- Naive Bayes provides simple probabilistic classification
- Bayesian networks model conditional dependencies via DAGs
- EM algorithm estimates parameters with hidden variables (e.g., mixture components)